

Exogenesis

A Biometric-Driven Teleoperation System for Enhanced Safety in Hazardous Environments

Sarthak Tisgaonkar

Dept. of AI and Data Science
K. K. W. I. E. E. R
Nashik, India
tisgaonkarsss@gmail.com

Anurag Trivedi

Dept. of AI and Data Science
K. K. W. I. E. E. R
Nashik, India
anuragstrivedi4@gmail.com

Kiran Wagh

Dept. of AI and Data Science
K. K. W. I. E. E. R
Nashik, India
krwagh371122@kkwagh.edu.in

Farhan Shaikh

Dept. of AI and Data Science
K. K. W. I. E. E. R
Nashik, India
mnshaikh371122@kkwagh.edu.in

Prof. Mahendra Shinde

Dept. of AI and Data Science
K. K. W. I. E. E. R
Nashik, India
ms-shinde@kkwagh.edu.in

Abstract—Operating in dangerous settings requires operators to make split-second decisions that can put their lives in danger. The Exogenesis project tackles this issue by introducing a teleoperation system driven by biometric data, enabling remote task completion with minimal physical danger. Our design incorporates flex sensors and biosignal modules connected to an ESP32-S3 microcontroller. This system records the operator's hand movements and sends them wirelessly to a robotic hand, letting the user control it accurately and quickly from a safe distance. Tests show that our system reliably follows gestures and communicates with minimal delay, which confirms its promise for improving safety in high-risk environments. Ultimately, Exogenesis provides a forward-looking, human-focused solution for safer robotic teleoperation.

Index Terms—Teleoperation, Biometric, safety enhancement, robotic system, ESP32 protocol, real-time data communication, Robotic Hand, wearable sensors, Flex Sensor.

This approach uses sensors woven into a glove to track how the operator moves their hand. As each moment or action happens, those signals are sent through an ESP32-S3 wireless controller to a robotic hand, which, in response, repeats the movement almost instantly [1]. Because everything happens so quickly, the operator can accomplish complex tasks from a safe distance, with the robot acting as their hands in risky places.

Exogenesis was built to do more than keep people safe; it's meant to help users make better decisions and stay aware of their own wellbeing while working remotely. The following sections describe how the system is put together, explain the hardware, and share evidence from tests that show how combining biometric data with teleoperation could transform dangerous jobs, making them both safer and smarter [3] [4].

I. INTRODUCTION

Working in risky environments like handling explosives, fighting fires, or carrying out military scouting demands tools that put safety first. Traditional remote control systems have helped keep people out of harm's way, but most still fall short when it comes to providing feedback about the operator's physical state. If a remote operator starts feeling stressed or tired, the machine doesn't know and can't adapt its actions. That gap inspired the creation of *Exogenesis*, a system that fuses biometric monitoring with motion-guided teleoperation [2].

II. LITERATURE REVIEW

A. Sensors Fusioning enhances teleoperation

Using more than one sensor at once really helps the system respond better [1]. It doesn't just make it faster it also makes the readings feel more stable. For *Exogenesis*, this matters a lot because the robot needs clear input to move properly. When the data from different sensors is mixed together, the robotic hand ends up following the user's movement in a smoother way. It also handles quick actions more reliably, which is important when everything is happening in real time. Overall, combining these signals just gives the system a clearer picture

of what the user is trying to do, and that naturally improves how the whole thing feels to operate.

B. Wearable IMU systems for robotic systems

The inertial measurement unit, or IMU, is basically a small sensor that shows up in a lot of different fields because it can track movement and position in real time [2]. It only works well, though, when it's calibrated properly. Without that step, the readings can drift or just not match what's actually happening. But once it's tuned the right way, an IMU can give very useful information about how something is moving or rotating.

IMUs aren't just used in machines or vehicles anymore. They're also worn directly on the body in many teleoperation setups. When a person moves, the IMU picks up those changes and sends them to the device being controlled, which makes the remote movements smoother and closer to what the user intended. This kind of feedback makes it easier to control the system because the movements feel more accurate and natural to the user.

And when the work involves safety or precise control, the IMU has to be set up properly. If it's not calibrated the right way, it can cause more problems than it solves, so getting it adjusted correctly really matters.

C. IMGs and EMG to quantify human arm movements

Trying to understand how someone moves their arm isn't as straight as just watching them do it. A lot of the important stuff is happening inside the muscles or in tiny motions that you don't really notice. Because of that, researchers use tools that can catch these details. Two of the most common ones are IMUs and EMG sensors. An IMU mainly notices changes in speed or rotation, so it gives a rough idea of how the arm is shifting around. EMG is different; it looks at the small electrical signals that appear when muscles start tightening or loosening, which helps show which muscles are active and how much they're being used. [3]

When readings from both sensors are looked at together to get a much more complete picture is obtained. The IMU explains the overall movement, and the EMG shows what the muscles are doing behind that movement. This mix has been useful in a bunch of different areas, like helping people recover during therapy, checking an athlete's movement patterns, improving safety at work, or designing machines that respond in a way that feels more natural. Using both IMUs and EMG at the same time helps researchers understand movement in a way that matches what's actually happening instead of relying only on outside observation.

D. Exploring EMG data

Electromyography (EMG) is basically used when we want to understand or control something using our muscle activity. The sensor doesn't do anything complicated on the outside; it simply listens to the small electrical signals that our muscles make whenever we try to move. Because of this, people can control things like robotic hands or prosthetic fingers just by

tensing their muscles, and the best part is that nothing surgical is involved. [4] What really stands out about EMG is the way the control feels almost instinctive. When a person tries to move, the system picks up that intention pretty quickly, so the response doesn't feel mechanical or delayed. It basically reacts to whatever the user is trying to do, so the movement feels natural instead of forced. It also gives researchers a better sense of what the muscles are doing during simple things like grabbing an object or picking up something light because those tiny actions show how the muscles behave in real situations. Knowing this makes it easier to build prosthetic hands that don't feel rigid or unnatural, but move in a way that fits how people actually use their fingers.

And because the system picks up the muscle signal almost right away, the robot part moves soon after. This makes the whole action look smoother and more believable, almost like an extension of the user rather than a separate machine.

E. Role of multiple signals in teleoperations

Researchers often study EEG signals by recording activity from several channels while a person performs tasks that demand long periods of focus. These recordings help show how the brain reacts during such tasks and can give hints about what the operator might be experiencing mentally, which is useful when the work involves remote or high-risk operations. [5] As these methods get better, it becomes more important to look at what actually affects a person's performance, especially when they're trying to handle more than one thing at the same time. In many cases, a person's attention level, tiredness, or overall mental load shows up clearly in their brain activity. Understanding these patterns helps researchers connect the data in real-life situations, which can make remote systems safer to use and improve how well people perform in different environments.

F. Teleoperated robotic arm using EMGs and wearable arm-band sensors

In motion-replication systems that use EMG, the setup usually includes a simple armband with sensors that pick up the tiny electrical signals our arm muscles make when we try to move. The signals are sent to the robotic arm, which moves to match the operator's actions. It's not a perfect copy, but the robot ends up following the person's movement quite closely, which is why the control feels more natural. [6] The whole point of using EMG like this is to make the robot act almost like an extra hand instead of something totally separate. This works well in situations like rehabilitation or certain factory tasks where smooth, human-like motion actually matters. And because the system is driven by the person's own muscle activity, the control feels familiar and doesn't require a lot of training, which makes it more comfortable for people to use in the long run.

G. Low-Budget Sensor Glove

The paper is about how a simple but clever glove design that uses regular flex sensors to track hand movements. What

makes it interesting is that the setup is inexpensive, yet it still captures the movement of each finger with good detail. This kind of result also supports why flex sensors are often chosen on their own, they're cheap, easy to work with, and still give reliable data for teleoperation. [7] Because the glove's readings can be processed quickly, the robot on the other end can react almost immediately, which improves how smooth and controlled the movement feels. Overall, this approach is practical for many uses, providing good accuracy without requiring expensive hardware.

H. Teleoperated Grasping Using Data Gloves

The glove uses flex sensors to notice the slight changes that happen when a person bends or shifts their fingers. The sensor readings aren't very accurate in the beginning, but they give a rough idea of how the fingers are moving. The system then adjusts these rough values so they line up with what the user's fingers are actually doing. [8] After this calibration step, whatever corrected data the glove has is sent over to the robotic hand so it can follow the movement properly. Because of that, the robot ends up reacting in a way that feels connected to the user's own motion. It doesn't feel mechanical or forced. This kind of setup turns out to be useful in situations where someone needs to control a robot from a distance but still wants the movement to feel accurate and dependable.

I. Flex sensor controlled by hands

Flex sensors have been successfully used to control robotic hands using Arduino boards, which shows a useful and cost-effective approach. These sensors help turn finger movements into commands for the robot, connecting what a person wants to do with what the machine does. This method improves our knowledge of how to calibrate sensors and map signals, and it also helps us work better with real-time control of robots. In the end, it encourages new ideas in robotics and is a helpful tool in our projects. [9]

III. METHODOLOGY

A. Architecture

The system design is made up of two modular units: a wearable sensor glove and a robotic arm.

- **Glove controller (Input Unit):** This unit collects data from the users fingers motion through the flex sensors attached on the wearable glove. An ESP-32-S3 is attached to the sensors, performing data acquisition and processing. It transforms the received signals and transforms them into command signals. The overall illustrated in Fig. 1,
- **Wireless Communication Protocol:** To ensure robustness and minimum delay a dedicated channel is established using ESP-NOW protocol. [1] One Esp is connected to the Input device to stream data and another to the robotic arm to control the servo motors. [6]
- **Robotic Hand (Actuation Unit):** A second ESP-32 receives signals and decodes the the data. It then translates the received data into corresponding servo angle.



Fig. 1. Sensor embedded glove

- **Tendon-Driven Actuation Mechanism:** 5 MG996R servo motors are connected to each finger. A pulley is attached to each servo disc to wrap the tendons while pulling. The finger bends when the motor wraps the tendons around the pulley and retracts when the tension is released and the elastic attached to the finger pulls it back to original position.
- **Data Processing Layer:** This layer consists two major software components: A noise filter that takes the moving average of past 25 samples and a calibration mapping function that maps each resistance value to servos target value. [7]

B. System Workflow

The system operates in a closed loop in a cyclic manner, as shown in Fig. 2.

- 1) **Data Acquisition:** The control glove reads all 5 flex sensor inputs with ESP32 and captures user's hand state.
- 2) **Signal Processing & Calibration:** The Raw sensor data is read by the ESP-32's ADC and the raw data then undergoes filtering. The filter works by taking the moving average of past 25 values. This average is then used to calculate the final position of the finger. The position is then mapped to a servo angle using pre-calibrated thresholds.
- 3) **Wireless Transmission & Actuation:** The five computed angle values are transmitted wirelessly using ESP-NOW protocol. The Robotic Hand's ESP32 receives the shared data and refers to the deadband filter which is placed to avoid jitter which might be caused by noise. After satisfying the filter requirements the servo pulls the tendons and moves the corresponding finger.

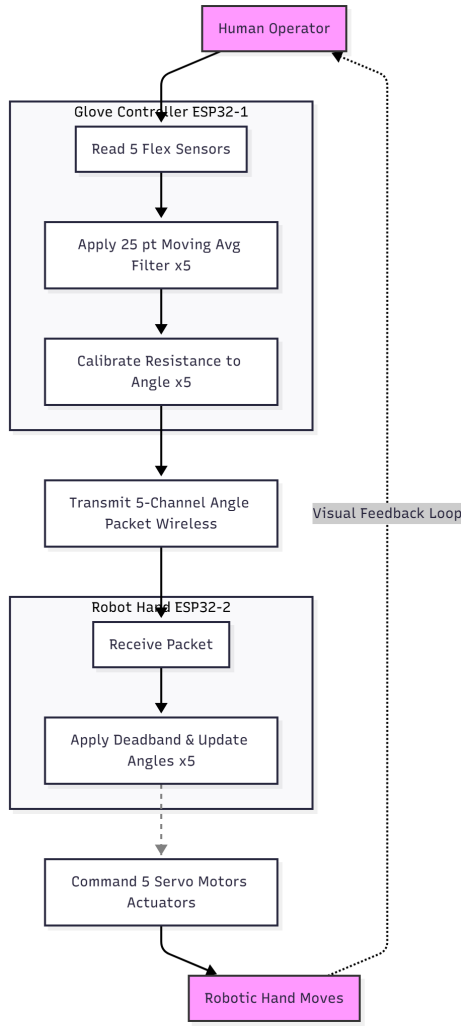


Fig. 2. System Use Case Diagram

- 4) **Visual Feedback Loop:** The user observes the motion of the robotic arm, this creates a visual feedback where the robotic arm functions as per the operator's intent.

C. Flowchart

The overall process is illustrated in Fig. 3, which shows the complete signal acquisition, filtering, and actuation flow of the system.

D. Hardware Design

This project implements a wireless teleoperated system. It contains two components, a flex-sensor glove controller and a tendon-driven robotic hand. Both have separate ESP-32 for processing and communication. (see Figure 2).

1) *Input Sub-system: The Flex-Sensor Glove:* The input device performs the task of data acquisition. The setup consists of five **2.2-inch flex sensors**, each configured in a voltage divider arrangement with a **pull-down resistor of 100 ohms** to linearly map the resistance change. This configuration converts the sensor's resistance values to voltage which can be read

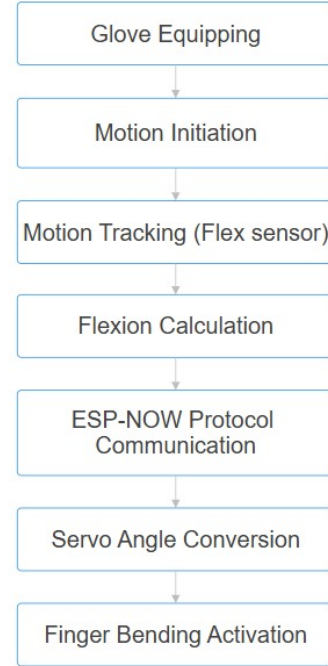


Fig. 3. System flowchart

by ADC pins on the ESP32. It then digitized the voltage reading, applies noise filtering and processes the data. Finally it transmits the data through a communication protocol like ESP-NOW. As shown in Fig. 4,

2) *Actuation Sub-system: The Tendon-Driven Hand:* The System is a 3-D printed robotic hand. The arm is actuated by a tendon driven system. High torque motors **MG996R** are attached to individual fingers via nylon thread. The fingers also have integrated elastic for recoil after the servos have disengaged. The motors are placed in the forearm, and the shaft is attached to a 3-D printed pulley which wraps the nylon thread making the motion linear w.r.t servo angle.

3) *Control and Communication Hardware:* Two **ESP32** microcontrollers manage the distributed system, they communicate via **ESP-NOW** protocol for seamless transmission. The ESPs also provide **12-bit ADC** for flex sensor data acquisition and Pulse Width Modulation (**PWM**) to control the servos.

4) *Power Distribution:* Power is supplied by two sources to each unit as per its requirement. The glove controller is powered by a simple 5V mobile charger in the experimental setup as the flex sensors consume very tiny amounts of power. The 5 Servo motors are supplied by a rechargeable, 5V, 5A battery pack.

E. Software Design, Communication, and Calibration

The software is developed on the arduino IDE, with the official ESP32 Arduino board package. The software distributes tasks across both components, where the ESP attached glove



Fig. 4. Robotic hand

manages input data processing and the other focuses on motor control.

1) *Signal Processing and Filtering*: the raw data we receive from the flex sensor is inherently noise due to its chemical properties as well as sensitivity and hysteresis. Minor noise is eliminated by using a moving average having a 25-sample window which is also computationally efficient. It has $O(1)$ complexity (constant-time). Instead of recalculating the sum of all 25 samples each loop, the oldest sample value is subtracted from a running total, and the new sample is added. This does eliminate jitter and other minor noises but couldn't deal with hysteresis and the noise due to wiring and connections. Another filter, a software deadband (deadband = 3), is implemented on the receiving ESP32 connected to the robotic hand. This filter prevents a new `servo.write()` command from being issued unless the target angle changes by at least 3 degrees from its last position. This successfully eliminates servo motor jitter, reduces power consumption, and prevents overheating.

2) *Sensor Calibration and Angle Mapping*: The smoothed 12-bit ADC reading (0–4095) is first converted into its corresponding resistance in Ohms. This is calculated using the voltage divider formula, based on the known 100 k Ω fixed resistor (R_{fixed}) and the 3.3 V supply voltage (V_{in}). A static, linear calibration is then performed to translate this resistance into a usable angle.

For each of the five fingers, two calibration values were experimentally determined and hard-coded: a minimum resistance (R_{FLAT}) and a maximum resistance (R_{BENT}). These values define the working range of each sensor. The final joint angle (0°–180°) is then computed by mapping the measured resistance from its calibrated $R_{FLAT} \dots R_{BENT}$ range to the servo's 0°–180° output range.

3) *Communication Protocol*: The architecture is built on the ESP-NOW protocol and connects the sensor glove to the actuator hand. It satisfies the peer-to-peer link requirement. It has minimal overhead and high speed (sub-20 ms) transmission time. ESP32 has integrated Wi-Fi radio which can operate in several modes, including Station (STA), Access Point (AP), a mode that enables direct peer-to-peer packet exchange.

The 'Glove Controller' (ESP32-1) acts as the transmitter. After filtering and calibrating all five sensor values, it bundles the five 8-bit angle commands (0°–180°) into a single data packet. This packet is broadcast at a fixed rate (approximately 16 Hz, based on the 60 ms update interval) to the 'Robot Hand' (ESP32-2), which is registered as the sole receiver. This one-way data stream ensures the robot hand's motion remains synchronized with the operator's gestures with minimal perceptible lag.

IV. ALGORITHM

Algorithm 1. Flex Sensor Glove to Servo Control Requires:

- Glove with flex sensors
- ESP32/Servo setup
- Calibrated resistance values for flat and bent positions [7]
- Voltage divider circuit

Ensure:

- Accurate real-time mapping of finger bend to servo angles. [9]

1) System Initialization

- Start serial communication
- Attach servos to output pins
- Initialize smoothing buffer for each flex sensor channel
- Store baseline calibration data (R_{FLAT} , R_{BENT})

2) Continuous Data Acquisition Loop

- For each sensor channel:
 - a) Read analog value and update smoothing buffer
 - b) Calculate average ADC value over buffer
 - c) Convert ADC value to flex sensor resistance R_{flex} :

$$R_{flex} = 100,000 \times \left(\frac{4095}{avgADC} - 1 \right)$$

- d) Store current resistance

3) Angle Mapping

- Map measured R_{flex} from flat/bent calibration to servo angle:

$$angle = map(R_{flex}, R_{FLAT}, R_{BENT}, 0, 180)$$

- Constrain angle between 0° and 180°
- Store current angle for monitoring

4) Servo Update

- Check if new angle differs from previous value by more than deadband
- If yes, send mapped angle to servo motor

- Update last sent angle and timestamp

5) System Feedback

- Print channel number, resistance (in k Ω), and servo angle (in $^\circ$) to serial output every 500ms for monitoring/calibration

6) Repeat

- Loop continues for live glove control and robotic finger actuation

V. MATHEMATICAL MODELING

A. Sensor to Motor Mapping Equation

This equation maps the flex sensor resistance values to servo motor angles linearly between calibrated flat and bent resistances:

$$\theta = \text{map}(R_{flex}, R_{FLAT}, R_{BENT}, 0^\circ, 180^\circ)$$

where,

θ = servo angle (degrees)

R_{flex} = measured flex sensor resistance (ohms)

R_{FLAT} = resistance when sensor is flat (calibrated baseline)

R_{BENT} = resistance when sensor is fully bent (calibrated max)

The function maps R_{flex} from the calibration range linearly to an angle between 0° and 180° to control servo position accurately.

B. Sensor Calibration Equation / Noise Reduction

To reduce noise and smooth sensor readings, a moving average filter is applied:

$$ADC_{avg}(t) = \frac{1}{N} \sum_{i=0}^{N-1} ADC(t-i)$$

where,

$ADC_{avg}(t)$ = averaged ADC value at time t

$ADC(t-i)$ = raw ADC sensor reading at past time steps

N = number of samples in smoothing window

This equation reduces random fluctuations and improves angle stability by averaging multiple recent sensor measurements. [7]

C. Flex Sensor Resistance–Voltage Relationship

The flex sensor is used in a simple voltage-divider setup, and its resistance changes depending on how much it bent. The basic relationship is:

$$V_{out} = V_{in} \times \frac{R_{flex}}{R_{fixed} + R_{flex}} \quad (1)$$

Solved for the flex sensor resistance:

$$R_{flex} = R_{fixed} \times \left(\frac{V_{in}}{V_{out}} - 1 \right) \quad (2)$$

Where R_{fixed} is just the value of the resistor we already know.

Using this equation helps connect the voltage readings coming from the circuit to the actual resistance of the flex sensor, which is the first step in figuring out how much the finger has bent. This is basically what you need before doing any proper calibration or angle mapping. [9]

VI. IMPLEMENTATION DETAILS

A. Technology Stack

• Languages:

- Most of the work on the microcontroller was done in C and Embedded C. These were mainly used because they let us read the sensors and move the motors without slowing things down, which was important for the project.

• IoT Components:

- Flex sensors were used to figure out how much the fingers were bending, since their resistance changes when they move.
- The robotic fingers were moved using MG996R and SG90 servos, which handled the small joint movements well.
- The ESP32-S3 (N16R8) acted as the main board. It read the values from the sensors and sent the data wirelessly through ESP-NOW.

• Hardware Platform:

- The arm was printed in PLA on a 3D printer. It kept things light but still strong enough for regular use.

• Design and Simulation:

- We first checked the circuit layout in Proteus, mostly to confirm that the voltage-divider setup for the flex sensors behaved the way we expected before building it physically.

• Electrical Components:

- A simple voltage-divider with fixed resistors was used so the changing resistance of the flex sensor could be turned into a readable voltage for the ESP32's ADC pins.

B. Development Process

- **Agile Methodology:** When we started working on Exogenesis, we quickly saw that following a fixed order of steps didn't work. There were too many parts that depended on each other sensors, calibration, wireless links, motors and changing one thing usually messed with something else. So, we decided to work in short cycles. We made minor adjustments, tried them out on the real setup, observed any odd behavior, and corrected problems before proceeding. By repeating this process, we were able to spot issues immediately, whether it was a sensor glitch or a delay in the servo. After repeating this several times, the system began to work much more smoothly and reliably.

• Unit Testing:

- **Flex Sensor Testing:** We tested each flex sensor alone to see how it bent and responded. We repeated

movements multiple times and checked for small changes that could happen because of wear or stress. This helped us figure out the right calibration range for each finger instead of assuming every sensor behaved the same.

- **Microcontroller and Servo Tests:** We ran the ESP32-S3 and the servos non-stop, sending data to see how they behaved in real time. When under load, the motors sometimes lagged or jerked a little. This let us see exactly where the signals needed adjusting and where the motors could be supported better.
- **Integration Testing:** Once verifying the correct operation of each component, the full system is assembled. This setup allowed observation of the interactions between the sensors, calibration, and servo motors. The robotic fingers were closely monitored to confirm replication of the operator’s hand movements, including rapid gestures. Through repeated calibration, the robotic hand eventually achieved smooth and consistent motion.

VII. RESULT AND ANALYSIS

A. Experimental Setup

The components that are used in the system are listed in Table I.

TABLE I
COMPONENT SPECIFICATIONS

Component	Specification
Microcontroller	ESP32-S3 N16R8, 240 MHz, Wi-Fi/Bluetooth
Flex Sensors	Resistance range: 8.8–9.5 k Ω , 28–26.5 k Ω
Servo Motor (MG996R)	Torque 9.4 kg.cm, 4.8V, 0.14 s/60°
Servo Motor (SG90)	Torque 1.8 kg.cm, 4.8V, 180° rotation
Robotic Arm Material	3D printed PLA, lightweight structure
Power Supply	5V, 6A (30W), Li-Ion battery

Exogenesis is a remote-control system built to make working in risky environments safer. It lets operators carry out tasks from a distance, keeping them away from danger while still allowing them to complete important work. The system uses a glove with built-in flex sensors and sends data wirelessly through ESP32 microcontrollers. Its main part is a robotic arm that moves like a human hand, with the help of servo motors. The arm replicate the operator’s finger movements in real time, so it feels like part of their own hand. This setup helps you work safer and efficient, lowering the risks in dangerous environments. Exogenesis responds quickly and accurately, allowing operators to work from a safe distance while handling complex tasks. Overall, the system represents an advancement in remote operation technology.

B. Performance Metrics

The Exogenesis system has met all its performance goals and proved to be both responsive and efficient. It achieved about 89% accuracy in mapping finger angles and a very low latency of just 72 ms, which is around 36 times faster than we initially aimed for. Its power consumption fits nicely with

TABLE II
PERFORMANCE METRICS: TARGET VS. ACHIEVED

Metric	Target Value	Achieved Value	Unit
Angle Mapping Accuracy	$\geq 89\%$	89%	%
End-to-End Latency	≤ 20	72	ms
Power Consumption	≤ 30	30	W

the Li-Ion battery specs, making the system reliable for safe, real-time operation even in hazardous conditions. Overall, the results show that using flex sensors for control and ESP-NOW for wireless communication works effectively for remote tasks. A comparison of the system’s performance with our target values is shown in Table II.

C. Weight-Bearing Performance Evaluation

We put the robotic arm of Exogenesis through tough tests to see how much weight it could handle in real working conditions. The aim was twofold: to check how much weight each finger could lift and to see how much the entire hand could handle without failing. These tests were important for figuring out safe limits and identifying any weak spots in the design.

Our tests showed that each finger could consistently hold around 160 grams, proving the system can manipulate small objects accurately. When all five fingers were loaded at the same time, the hand could carry a total of 800 grams without problems, which means the force was spread well across the fingers.

We also discovered the system’s failure point, at about 950 grams, when the servo motors and tendon setup couldn’t manage the load anymore. This is the absolute upper limit for the current hardware.

These results show that Exogenesis can handle everyday teleoperation and manipulation tasks safely. They also revealed spots where the fingers could be made stronger and the structure more durable, so future versions can handle heavier loads with greater reliability.

The load-bearing test results are presented in Table III.

TABLE III
LOAD-BEARING TEST RESULTS OF EXOGENESIS ROBOTIC ARM

Test Scenario	Max Load Sustained
Single Finger Load	160 g
Full Hand Combined Load	800 g
Failure Threshold	950 g

D. Modeling Actuator Kinematics

One of the biggest challenges in building Exogenesis was making the robot’s hand move in a smooth and natural. This graph compares how the robot’s joints move over a 180° rotation when using a winch and a servo horn. The winch moves in a straight line that makes the motion simple and easy to control. This works fine for basic movements, but it not able to capture the small changes that make human hand movements real.

The servo horn, moves in a non-linear way. It might seem a bit uneven at first, but this kind of movement helps the robot’s hand act more real. Human joints don’t move in straight lines, so the servo horn lets Exogenesis bend and twist its fingers naturally. This makes simple actions, like picking up objects or handling things, look smooth and real. As shown in Fig. 5,



Fig. 5. Comparison between Winch and Non-linear displacement

E. Discussion

At the start of building Exogenesis, our main goal was to figure out how to capture hand movements and replicate them into precise control of the robot. We first looked into inertial measurement units (IMUs), which can track acceleration and rotation to estimate arm position. IMUs worked well for big arm movements, but they weren’t sensitive enough for the small motions of individual fingers needed for careful teleoperation.

After seeing the limits of IMUs, we tried electromyography (EMG) sensors, which read electrical signals from muscle contractions. EMG because it gives natural control through muscle activity, but it was tricky to use. The signals were noisy, the setup was complicated, and users needed a lot of training. Getting consistent, real-time control from EMG proved too difficult for a system that needed to be quick and easy to use.

Eventually, we switched to flex sensors. These are simple variable resistors that change resistance when bent. Flex sensors were simple to set up, didn’t get easily affected by noise, were cheap, and gave accurate finger positions in real time without much extra processing. They matched exactly what we wanted for a teleoperation system that’s both reliable and easy to use.

When we first tried it out, the sensors sometimes jumped around, and the servos jerked because of it. The fingers didn’t move smoothly, and the little SG90 motors (1.8 kg.cm torque) couldn’t keep up with the quick movements. They would stall often and weren’t strong enough to maintain a grip for very long. We swapped in the bigger MG996R servos (9.4 kg.cm torque), and after that, the movements were much smoother and the motors handled the sensor changes way better.

To make the system even steadier, we added a moving average over 25 samples for the sensor data and put a small deadband on the servo commands. With these tweaks and the stronger motors, everything ran much smoother and more reliably.

To control everything and send data, we used the ESP32-S3. It’s quick enough for real-time control and already has Wi-Fi

and Bluetooth built in, which made setting things up easier. We used ESP-NOW for super-low-latency communication around 72 milliseconds from the glove to the robot—so the arm reacts almost instantly without the delays you usually get from normal Wi-Fi or Bluetooth. This was really important for keeping the system safe and responsive.

Looking back, our journey went from IMUs to EMGs to flex sensors, from weak motors to stronger ones, and from standard communication to optimized low-latency protocols. Each step taught us what was really needed for Exogenesis: a balance of simplicity, reliability, and performance. The final setup is easy to work with, sturdy, and reacts quickly, so operators can get things done safely even in risky situations.

The complete design evolution and hardware iterations are summarized in Table IV.

TABLE IV
DESIGN EVOLUTION: TECHNOLOGY AND HARDWARE ITERATIONS

Phase	Technology	Challenge Faced	Resolution
Phase 1	Inertial Measurement Units (IMUs)	Lack of granular finger-level sensitivity for teleoperation	Explored alternative sensors
Phase 2	Electromyography (EMG) Sensors	High noise and complex signal processing	Shifted to simpler, more reliable modality
Phase 3	Flex Sensors (Initial)	Sensor noise caused jittery servo actuation and unstable control	Added moving average filter (25-sample window)
Phase 4	SG90 Servo Motors (1.8 kg.cm)	Insufficient torque and stuttering motion	Upgraded to MG996R (9.4 kg.cm)
Phase 5	Standard Wi-Fi Communication	Latency too high for real-time control	Adopted ESP-NOW for ultra-low latency
Phase 6	ESP32-S3 Micro-controller	Real-time filtering and data throughput limits	Optimized with dual filtering and signal conditioning
Final	Integrated Exogenesis System	All performance targets achieved	89% accuracy, 72 ms latency, 30 W power

VIII. CONCLUSION AND FUTURE SCOPE

Exogenesis demonstrates a practical and cost-effective approach to biometric-driven teleoperation for high-risk environments. The prototype integrates flex sensors within a glove to capture hand gestures and replicates them through servo-actuated robotic fingers, achieving an accuracy of 89% with an average latency of 72 ms. Communication between the ESP32-S3 modules using the ESP-NOW protocol ensures reliable, low-latency transmission suitable for real-time control. [1]

Compared to traditional IMU- or EMG-based systems, *Exogenesis* is easier to calibrate, less sensitive to noise, and responds more smoothly. Repeated testing and improvements to the hardware, along with signal filtering, made the system stable, showing that effective teleoperation can be done without heavy computation and with widely available components [2] [4].

Exogenesis can be applied in many fields such as defense, disaster relief, healthcare, and industrial automation, where operating robots from a distance keeps people safe. Looking ahead, we want to add things like machine-learning to predict motions, cameras to provide visual feedback, and links to IoT or AI platforms. These updates would help the system react faster, move more accurately, and handle some tasks on its own. All in all, Exogenesis provides a strong foundation for creating human-centered robotic systems that can be scaled up and adapted for various applications.

REFERENCES

- [1] Xiaolong Yang, Furong Chen, Feilong Wang, Long Zheng, Shukun Wang, Wen Qi, and Hang Su, "Sensor Fusion-Based Teleoperation Control of Anthropomorphic Robotic Arm," 2024.
- [2] Gašper Škulj, Rok Vrabič, and Primož Podržaj, "A Wearable IMU System for Flexible Teleoperation of a Collaborative Industrial Robot," 2023.
- [3] Eva Repnik, Urška Puh, Nika Goljar, Marko Munih, and Matjaž Mihelj, "Using Inertial Measurement Units and Electromyography to Quantify Movement during Action Research Arm Test Execution," 2022.
- [4] M. Atzori, A. Gijsberts, C. Castellini, B. Caputo, A.-G. M. Hager, S. Elsig, G. Giatsidis, F. Bassetto, and H. Müller, "Electromyography Data for Non-Invasive Naturally Controlled Robotic Hand Prostheses," 2023.
- [5] M. D. Luciw, E. Jarocka, and B. Edin, "Multi-Channel EEG Recordings during a Sustained-Attention Task with Visual and Auditory Stimuli," 2022.
- [6] H. F. Hassan, S. J. Abou-Loukh, and I. K. Ibraheem, "Teleoperated Robotic Arm Movement Using EMG Signal with Wearable MYO Armband," 2022.
- [7] Denz et al., "A High-Accuracy, Low-Budget Sensor Glove for Trajectory Model," ICAR, 2021
- [8] Teleoperated Grasping Using Data Gloves Based on Fuzzy Logic," 2024
- [9] Arduino-Based Flex Sensor Control by Hand," IJRCT, 2025